

Mini-projet S9 : Pyxel

1. Exercice 1

L'objectif de ce mini-projet est de créer le célèbre mastermind (en version console). Il y a plusieurs fonctions à écrire, qu'il vous faudra vous répartir.



a) Variable globale des couleurs

Dans le code, on va définir l'ensemble des couleurs que l'on pourra jouer. Il faudra définir une variable au début du programme comprenant les couleurs à jouer. Enregistrer sous mastermind.py

Réponse :

b) Génération d'un code

Le fonction « **genere_jeu(nombre couleur, taille)** » qui prend en paramètre un nombre de couleurs à utiliser (on est pas obligé d'avoir un code avec toutes les couleurs), et qui génère une combinaison en fonction de la taille. On stockera les couleurs choisies dans une liste, qu'on renverra. Enregistrer sous mastermind.py

Le choix des couleurs sera évidemment aléatoire.

Réponse :

c) Saisie des couleurs

La fonction « **saisie_utilisateur(taille)** » demandant à l'utilisateur de saisir une couleur au clavier, l'ajoute dans une liste, et retourne la liste constituée de l'ensemble des couleurs de l'utilisateur. Enregistrer sous mastermind.py

Réponse :

d) Placement des bonnes couleurs

La fonction « **couleurBienPlace(jeu, saisie, tab)** » qui a comme paramètres 3 listes : le code généré, le code du joueur, ainsi que résultat, une liste permettant d'afficher à l'utilisateur le résultat du placement des jetons. Enregistrer sous mastermind.py

On va vérifier dans cette fonction si des pions du joueur sont positionnés au même endroit que les pions de l'ordinateur. Si c'est le cas, on va mettre le caractère 'B' (bien) dans la liste resultat, à la même position.

Réponse :

e) Placement des couleurs mal positionnées

La fonction « **couleurMalPlace(jeu, saisie, tab)** ». Même chose que la fonction précédente, cette fois-ci on va mettre la valeur 'M' quand une couleur sera mal placée. Enregistrer sous mastermind.py

Une couleur mal placée est une couleur existante dans le code généré que l'utilisateur a mal placé dans son jeu.

Mini-projet S9 : Pyxel

Réponse :

Exemple :

code généré : rouge, vert, bleu

code utilisateur : vert, jaune, bleu

resultat obtenu : 'M', 0, 'B' car le vert est mal positionné, le jaune n'existe pas, et le bleu est bien positionné.

f) Résultat des placements de couleurs

La fonction « **nbJetonPlace(jeu, saisie)** » qui crée une liste de résultat initialisée avec des 0, indique les couleurs bien placées, celles mal placées, et retourne la liste finale.

Enregistrer sous mastermind.py

Réponse :

g) Le jeu

La fonction « **jeu()** » qui est le jeu principal. Voici le déroulé :

1. On définit un nombre d'essai, les couleurs à utiliser, ainsi que la taille du code.

2. On génère le code de l'ordinateur.

3. On propose à l'utilisateur de saisir tant que le code n'est pas trouvé ou qu'il nous reste des essais.

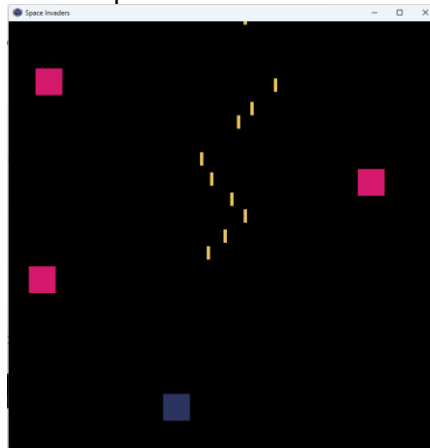
4. On détermine et affiche la position des couleurs du joueur, le joueur a gagné si toutes les valeurs de la liste resultat sont 'B'.

Enregistrer sous mastermind.py

Réponse :

2. Exercice 2

L'objectif est de créer une sorte de mini space-invader.



Créer une fenêtre de taille 128x128px, ainsi qu'un carré de taille 8px, de n'importe quelle couleur.

Enregistrer sous space_invaders.py

Réponse :

Écrire une fonction **deplacement()** permettant de faire se déplacer le carré en appuyant sur les flèches, modifiant ainsi ses coordonnées. Vous utiliserez une variable x et y, qui sera globale, et initialisée au début du programme en bas et au milieu.

Enregistrer sous space_invaders.py

Réponse :

Mini-projet S9 : Pyxel

Nous allons faire tirer notre petit carré dans les questions suivantes.

Créer une liste **tirs**, vide, au début du programme.
Écrire une fonction **creer_tirs()** qui permet, lorsque l'on appuie sur la barre espace, d'ajouter dans la liste tirs une liste avec comme éléments la position en x et en y du tir (concrètement, on crée un tir qui se place devant notre carré).
Appeler cette fonction dans la fonction update().
Enregistrer sous space_invaders.py

Réponse :

Dans la fonction draw(), créer pour chaque tir de la liste tirs un rectangle d'1px d'épaisseur, et 4px de hauteur, avec la couleur numéro 10.
Enregistrer sous space_invaders.py

Réponse :

Écrire une fonction **deplacement_tirs()** qui parcourt l'ensemble des tirs de la liste tirs, et change leur coordonné y pour faire avancer les tirs. Les tirs vont vers le haut. Les tirs seront supprimés s'ils sortent de la fenêtre.
Appeler cette fonction dans update().
Enregistrer sous space_invaders.py

Réponse :

Nous allons faire la même chose pour créer des ennemis.

Créer une liste **ennemis**, vide, au début du programme.
Écrire une fonction **ennemis_creation()** qui crée un ennemi toutes les secondes (utilisez `pyxel.frame_count%30`).
La fonction ajoutera dans la liste **ennemis** une liste avec des points de coordonnées, dont x sera généré aléatoirement.
Appeler cette fonction dans update().
Enregistrer sous space_invaders.py

Réponse :

Dans la fonction draw(), créer pour chaque ennemi de la liste ennemis un carré de 8px, avec la couleur numéro 8.
Enregistrer sous space_invaders.py

Réponse :

Écrire une fonction **ennemis_deplacement()** qui fait la même chose que **deplacement_tirs()**, mais les ennemis vont vers le bas.
Appeler cette fonction dans update().
Enregistrer sous space_invaders.py

Réponse :

Nous allons faire les collisions maintenant.

La collision se fait lorsque la coordonnée en x d'un tir est comprise dans les dimensions d'un ennemi. Ainsi, si un tir est en coordonnées $x = 8$ $y = 10$, et qu'un ennemi est en coordonnées $x = 5$ $y = 11$, le tir touche car l'ennemi a une taille de 8x8.

Mini-projet S9 : Pyxel

Écrire une fonction **ennemis_suppression()** qui permet de supprimer un tir et un ennemi si l'une des coordonnées d'un tir est comprise dans les coordonnées d'un ennemi.

Écrire une fonction **vaisseau_suppression()** qui fait la même chose que la fonction précédente, sauf que l'on va vérifier la collision du carré avec un ennemi. On utilisera une nouvelle variable vie initialisée à 5, et cette valeur descendra s'il y a collision.

Appeler ces 2 fonctions dans update().
Enregistrer sous space_invaders.py

Réponse :

Dans draw(), afficher un texte dans le coin supérieur gauche qui affiche le nombre de vies restante. On effacera l'écran et affichera « game over » si la vie est égale à 0.
Enregistrer sous space_invaders.py

Réponse :

Ajouter un score en fonction du nombre de carrés détruits, et l'afficher dans le coin supérieur droit.
Enregistrer sous space_invaders.py

Réponse :

Si cela vous a plu et que vous en redemandez, vous pouvez ajouter des bonus aléatoires ! Modifier le système de tirs (temps de recharge de munitions etc), ajouter des boss, des points de vie aux vaisseaux.